

LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 13

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, struktur kode yang baik sangat penting agar aplikasi mudah dibaca, dipelihara, diuji, dan dikembangkan. Pada aplikasi sederhana, Controller masih dapat digunakan untuk menangani berbagai proses sekaligus, seperti menerima request, melakukan validasi data, menjalankan proses bisnis, hingga menyimpan data ke database. Namun, apabila aplikasi semakin besar, pendekatan tersebut dapat menyebabkan Controller memiliki terlalu banyak tanggung jawab.

Pada Sistem Informasi Akademik, proses penambahan dan perubahan data mahasiswa tidak hanya membutuhkan penyimpanan data ke database. Sebelum data disimpan, aplikasi perlu memastikan bahwa NIM telah diisi, NIM belum digunakan oleh mahasiswa lain, email memiliki format yang valid, program studi tersedia, serta data lainnya memenuhi ketentuan aplikasi.

Apabila seluruh proses tersebut diletakkan di dalam Controller, kode menjadi panjang dan sulit dipelihara. Oleh karena itu, diperlukan sebuah lapisan tambahan yang bertanggung jawab khusus terhadap logika bisnis aplikasi, yaitu **Service Layer**.

Service Layer ditempatkan di antara Controller dan Repository. Controller bertugas menerima request dari pengguna dan menentukan response, Service bertugas menjalankan validasi dan logika bisnis, sedangkan Repository bertugas melakukan akses data melalui query database. Dengan pembagian tugas tersebut, setiap class memiliki tanggung jawab yang lebih jelas.

Pada praktikum ini juga digunakan **Flash Message** menggunakan session. Flash Message digunakan untuk memberikan informasi sementara kepada pengguna setelah suatu proses selesai, misalnya ketika data berhasil ditambahkan, berhasil diperbarui, atau ketika proses gagal karena NIM sudah digunakan. Pesan tersebut hanya ditampilkan satu kali kemudian dihapus dari session.

Melalui penerapan Service Layer dan Flash Message pada Sistem Informasi Akademik, struktur aplikasi menjadi lebih terorganisasi dengan alur Controller menuju Service, kemudian Repository, dan akhirnya Database.

1.2 Tujuan Praktikum

Praktikum Acara 13 bertujuan agar mahasiswa:

1. Memahami konsep Service Layer dalam pengembangan aplikasi berbasis PHP.
2. Memahami pembagian tanggung jawab antara Controller, Service, Repository, dan Database.

3. Mampu membuat class MahasiswaService.
4. Mampu menerapkan Dependency Injection pada Service Layer.
5. Mampu memindahkan proses validasi data mahasiswa dari Controller ke Service.
6. Mampu membuat proses penambahan data melalui method create() pada Service.
7. Mampu membuat proses perubahan data melalui method update() pada Service.
8. Mampu menjaga agar Controller tetap sederhana dan tidak menangani logika bisnis secara langsung.
9. Mampu menggunakan Repository sebagai lapisan akses database.
10. Mampu menerapkan Flash Message menggunakan session.
11. Mampu menampilkan pesan keberhasilan maupun kegagalan proses create dan update.
12. Memahami pentingnya pemisahan alur request, logika bisnis, dan akses database ketika aplikasi semakin kompleks.

Bab 2. Dasar Teori

2.1 Service Layer

Service Layer merupakan lapisan aplikasi yang digunakan sebagai tempat menjalankan logika bisnis. Service berada di antara Controller dan Repository. Pada pola tersebut, setiap bagian memiliki tanggung jawab yang berbeda.

Controller bertanggung jawab menerima request pengguna dan menentukan response. Service bertugas mengatur aturan bisnis dan validasi. Repository bertanggung jawab melakukan komunikasi dengan database.

Sebagai contoh ketika pengguna menambahkan mahasiswa, Controller tidak perlu mengetahui bagaimana cara mengecek NIM duplikat. Controller hanya menyerahkan data kepada Service. Selanjutnya Service melakukan validasi dan menggunakan Repository apabila membutuhkan informasi dari database.

2.2 Controller

Controller merupakan bagian aplikasi yang menangani request dari pengguna. Pada arsitektur yang menggunakan Service Layer, Controller sebaiknya dibuat sesederhana mungkin. Controller biasanya tidak menjalankan query database maupun validasi secara langsung. Tugas tersebut diberikan kepada Service dan Repository.

2.3 Repository

Repository merupakan lapisan yang bertanggung jawab terhadap akses data. Semua operasi yang berhubungan dengan database dapat ditempatkan pada Repository. Dengan Repository, Service tidak perlu mengetahui sintaks SQL yang digunakan.

2.4 Dependency Injection

Dependency Injection merupakan teknik memberikan object yang dibutuhkan sebuah class melalui bagian luar class tersebut. Dependency Injection membuat hubungan antarclass menjadi lebih jelas dan kode lebih mudah dikembangkan.

2.5 Validasi sebagai Logika Bisnis

Validasi merupakan proses memeriksa apakah data yang diterima telah sesuai dengan aturan aplikasi.

Pada praktikum ini beberapa validasi mahasiswa adalah:

- NIM wajib diisi
- NIM harus berupa angka
- Nama wajib diisi
- Email harus valid

- Tahun angkatan harus valid
- NIM harus sesuai tahun angkatan
- NIM tidak boleh digunakan dua mahasiswa
- Program studi harus tersedia

Validasi tersebut ditempatkan pada MahasiswaService. Dengan demikian Controller tidak perlu mengetahui aturan bisnis tersebut.

2.6 Flash Message

Flash Message merupakan pesan sementara yang disimpan menggunakan session. Pesan biasanya digunakan setelah redirect agar pengguna mengetahui hasil suatu proses.

2.7 Session

Session digunakan untuk menyimpan data sementara pada server selama pengguna menggunakan aplikasi. Session dimulai dengan `session_start()`. Pada praktikum ini session digunakan sebagai tempat penyimpanan Flash Message.

Bab 3. Hasil Praktikum

3.1 Kondisi Awal Aplikasi

Sebelum dilakukan refactoring, proses validasi data mahasiswa masih berada di dalam MahasiswaController.

Controller melakukan proses seperti:

- mengambil input form
- memeriksa NIM
- memeriksa nama
- memeriksa email
- memeriksa tahun angkatan
- memeriksa kesesuaian NIM dan angkatan
- menyimpan data menggunakan Repository

Hal tersebut menyebabkan Controller memiliki tanggung jawab terlalu banyak.

Oleh karena itu, dilakukan refactoring dengan menambahkan Service Layer.

3.2 Pembuatan Prodi Repository

Repository program studi dibuat pada app/Repositories/ProdiRepository.php. Repository tersebut menyediakan proses pencarian program studi dan pengecekan keberadaan program studi.

```
68     public function exists(  
69         int $id  
70     ): bool {  
71  
72         $stmt = $this->db->prepare("  
73             SELECT COUNT(*)  
74             FROM prodi  
75             WHERE id = :id  
76         ");  
77  
78         $stmt->execute([  
79             'id' => $id  
80         ]);  
81  
82         return  
83             (int) $stmt->fetchColumn() > 0;  
84     }
```

Method tersebut digunakan Service untuk memastikan program studi yang dipilih pengguna benar-benar tersedia.

3.3 Penambahan Pengecekan NIM

Pada MahasiswaRepository ditambahkan method existsByNim(). Method tersebut digunakan untuk mengetahui apakah NIM telah digunakan mahasiswa lain. Parameter \$excludeId digunakan ketika melakukan update sehingga NIM milik mahasiswa yang sedang diedit tidak dianggap sebagai duplikat dirinya sendiri.

```
236 public function existsByNim(  
237     string $nim,  
238     ?int $excludeId = null  
239 ): bool {  
240  
241     $sql = "  
242         SELECT COUNT(*)  
243         FROM mahasiswa  
244         WHERE nim = :nim  
245     ";  
246  
247     if ($excludeId !== null) {  
248         $sql .= "  
249             AND id != :id  
250         ";  
251     }  
252  
253     $stmt = $this->db->prepare($sql);  
254  
255     $params = [  
256         'nim' => $nim  
257     ];  
258  
259     if ($excludeId !== null) {  
260         $params['id'] = $excludeId;  
261     }
```

3.4 Pembuatan Mahasiswa Service

Service menerima dua dependency:

MahasiswaRepository digunakan untuk mengakses data mahasiswa, sedangkan ProdiRepository digunakan untuk mengakses data program studi.

Penggunaan constructor tersebut menunjukkan penerapan Dependency Injection.

```
10
    5 references | 0 implementations
11  class MahasiswaService
12  {
13      5 references
      3 references
      private MahasiswaRepository $repo;
14      private ProdiRepository $prodiRepo;
15
      1 reference | 0 overrides
16      public function __construct(
17          MahasiswaRepository $repo,
18          ProdiRepository $prodiRepo
19      ) {
20          $this->repo = $repo;
21          $this->prodiRepo = $prodiRepo;
22      }
```

3.5 Implementasi Method Create

```
$this->repo->create($mahasiswa);

return [
    'success' => true,
    'message' => 'Data mahasiswa berhasil ditambahkan.'
];

} catch (InvalidArgumentException $e) {

    return [
        'success' => false,
        'message' => $e->getMessage()
    ];

} catch (PDOException $e) {

    return [
        'success' => false,
        'message' => 'Data mahasiswa gagal ditambahkan.'
    ];

}
```

3.6 Implementasi Method Update

```
97         $this->repo->update(  
98             $id,  
99             $mahasiswa  
00         );  
01  
02         return [  
03             'success' => true,  
04             'message' => 'Data mahasiswa berhasil diperbarui.'  
05         ];  
06  
07     } catch (InvalidArgumentException $e) {  
08  
09         return [  
10             'success' => false,  
11             'message' => $e->getMessage()  
12         ];  
13  
14     } catch (PDOException $e) {  
15  
16         return [  
17             'success' => false,  
18             'message' => 'Data mahasiswa gagal diperbarui.'  
19         ];  
20     }
```

3.7 Pemindahan Validasi ke Service

```
73  
74     $mahasiswa = $this->validate($data);  
75  
76     if (  
77         $this->repo->existsByNim(  
78             $mahasiswa->getNim(),  
79             $id  
80         )  
81     ) {  
82         throw new InvalidArgumentException(  
83             'NIM sudah digunakan oleh mahasiswa lain.'  
84         );  
85     }  
86
```

2 references

```
124 private function validate(array $data): Mahasiswa
125 {
126     $nim = trim($data['nim'] ?? '');
127     $nama = trim($data['nama'] ?? '');
128     $email = trim($data['email'] ?? '');
129     $prodiId = (int) ($data['prodi_id'] ?? 0);
130     $angkatan = (int) ($data['angkatan'] ?? 0);
131
132     if ($nim === '') {
133         throw new InvalidArgumentException(
134             'NIM wajib diisi.'
135         );
136     }
```

```
144     if ($nama === '') {
145         throw new InvalidArgumentException(
146             'Nama wajib diisi.'
147         );
148     }
149
150     if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL))
151         throw new InvalidArgumentException(
152             'Format email tidak valid.'
153         );
154     }
155
156     if ($prodiId <= 0) {
157         throw new InvalidArgumentException(
158             'Program studi wajib dipilih.'
159         );
160     }
161
162     if ($angkatan < 2000 || $angkatan > 2100) {
163         throw new InvalidArgumentException(
164             'Tahun angkatan tidak valid.'
165         );
166     }
```

3.8 Perubahan Controller

Setelah Service Layer diterapkan, method `store()` menjadi lebih sederhana. Controller hanya menerima hasil dari Service kemudian menentukan apakah pengguna diarahkan ke halaman daftar mahasiswa atau dikembalikan ke halaman tambah data. Dengan perubahan tersebut Controller tidak lagi menangani validasi secara langsung.

```
153 public function store(): void
154 {
155     $result =
156         $this->service->create(
157             $_POST
158         );
159
160
161     if ($result['success']) {
162
163         $this->setFlash(
164             $result['message'],
165             'success'
166         );
167
168
169         $this->redirect(
170             '/mahasiswa'
171         );
172
173         return;
174     }
175
176
177     $this->setFlash(
178         $result['message'],
```

```

0 references | 0 overrides
285     public function update(
286         $id
287     ): void {
288
289         $id = (int) $id;
290
291
292         $result =
293             $this->service->update(
294                 $id,
295                 $_POST
296             );
297
298
299         if ($result['success']) {
300
301             $this->setFlash(
302                 $result['message'],
303                 'success'
304             );
305
306
307             $this->redirect(
308                 '/mahasiswa'
309             );
310

```

3.9 Dependency Injection pada public/index.php

Dependency dibuat pada public/index.php. Pertama dibuat object Repository

```

143     $mahasiswaRepository =
144         new MahasiswaRepository(
145             $database
146         );
147
148     $prodiRepository =
149         new ProdiRepository(
150             $database
151         );
152

```

Kemudian Repository diberikan kepada Service:

```

154     $mahasiswaService =
155         new MahasiswaService(
156             $mahasiswaRepository,
157             $prodiRepository
158         );
159

```

Selanjutnya Service diberikan kepada Controller melalui factory.

```
170         return new MahasiswaController(  
171             $mahasiswaRepository,  
172             $prodiRepository,  
173             $mahasiswaService  
174         );  
175     },
```

Hal tersebut menunjukkan penerapan Dependency Injection dengan alur dependency yang terstruktur.

3.10 Implementasi Flash Message

Flash Message menggunakan session yang telah tersedia pada BaseController Penyimpanan pesan dilakukan melalui

```
360         $this->setFlash(  
361             'Data mahasiswa berhasil dihapus.',  
362             'success'  
363         );  
364  
365     } catch (PDOException $e) {  
366  
367         $this->setFlash(  
368             'Data mahasiswa gagal dihapus.',  
369             'danger'  
370         );
```

Setelah pesan dibaca menggunakan getFlash(), data tersebut dihapus dari session menggunakan:

```
unset($_SESSION['flash']);
```

Dengan demikian pesan hanya tampil satu kali.

3.11 Pengujian Penambahan Data

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data mahasiswa berhasil ditambahkan.

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	251004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	SI - Sistem Informasi	2025	Aktif	DetailEditHapus
2	250004	Fazio Yusuf	fazioyusuf7@gmail.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus
3	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	DetailEditHapus
4	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus
5	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus

3.12 Pengujian NIM Duplikat

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

NIM sudah digunakan.

Tambah Data Mahasiswa

NIM

Nama Mahasiswa

Email

Program Studi

-- Pilih Prodi --

Angkatan

3.13 Pengujian Update

Edit Data Mahasiswa

NIM

250004

Nama Mahasiswa

Fazio Yusuf

Email

fazloyusuf7@gmail.com

Program Studi

TI - Teknik Informatika

Angkatan

2025

Status Mahasiswa

Aktif

Kembali

Simpan Perubahan

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data mahasiswa berhasil diperbarui.

3.14 Pengujian Tahun Angkatan Tidak Sesuai dengan NIM

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Dua digit pertama NIM harus sesuai dengan tahun angkatan.

Edit Data Mahasiswa

NIM

251004

Nama Mahasiswa

Nila Agustin Prima Aurelia

Email

nilaaurelia97@gmail.com

Program Studi

SI - Sistem Informasi

Angkatan

3.15 Pengujian Hapus Data

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data mahasiswa berhasil dihapus.

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	251004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	SI - Sistem Informasi	2025	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 4 dari 4 data mahasiswa.

Bab 4. Kesimpulan

Berdasarkan praktikum Acara 13, dapat disimpulkan bahwa Service Layer memiliki peran penting dalam menjaga struktur aplikasi agar tetap terorganisasi ketika logika bisnis semakin kompleks. Penggunaan Service Layer memungkinkan proses validasi dan aturan bisnis dipisahkan dari Controller sehingga Controller hanya bertanggung jawab menerima request, memanggil Service, dan menentukan response.

Pada Sistem Informasi Akademik, MahasiswaService digunakan untuk menangani proses create dan update serta validasi data seperti NIM wajib diisi, pengecekan NIM duplikat, format email, tahun angkatan, kesesuaian NIM dengan tahun angkatan, dan keberadaan program studi. Sedangkan proses query database tetap ditempatkan pada MahasiswaRepository dan ProdiRepository.

Dependency Injection digunakan untuk memberikan Repository kepada Service serta memberikan Service kepada Controller. Pendekatan ini menyebabkan hubungan antarclass menjadi lebih jelas dan mengurangi ketergantungan langsung antarbagian aplikasi.

Selain Service Layer, praktikum juga menerapkan Flash Message berbasis session. Flash Message memberikan informasi hasil proses kepada pengguna, seperti data berhasil ditambahkan, data berhasil diubah, data gagal disimpan, atau NIM telah terdaftar. Pesan tersebut hanya ditampilkan satu kali karena data session langsung dihapus setelah dibaca.

Dengan demikian, penerapan pola:

Controller → Service → Repository → Database

membuat aplikasi Sistem Informasi Akademik lebih rapi, mudah dipahami, mudah dipelihara, dan lebih mudah dikembangkan ketika kebutuhan aplikasi menjadi semakin kompleks.